```
-- Calculate the average completion time by category
SELECT category,
       avg(date_completed - date_created) AS completion_time
  FROM evanston311
 GROUP BY category
 ORDER BY completion_time DESC;
```

- **Date/Time Data Types**: You discovered that PostgreSQL handles date/time data using three main types: date, timestamp, and interval. Dates include the year, month, and day. Timestamps extend dates with time information, and intervals represent durations.
- **ISO 8601 Standard**: You learned about the importance of the ISO 8601 standard for recording date/time information. This standard helps in avoiding ambiguity by specifying a clear format: YYYY-MM-DD for dates and YYYY-MM-DD HH:MM:SS for timestamps, with time zones expressed as offsets from UTC.
- **Date Comparisons and Arithmetic**: You explored how to compare dates and perform arithmetic operations with them. For example, adding or subtracting dates to calculate intervals, and using the now() function to get the current timestamp.
- **Practical Example with SQL**: You practiced these concepts through SQL queries, such as calculating the average completion time for requests in a dataset by subtracting the date_created from the date_completed timestamp.

TRUNCATE, EXTRACT, DATE_PART

- **Date/time components**: Understanding how to extract specific parts of a date or timestamp, such as year, month, day, hour, minute, and second, using functions like date_part and extract. These components help in analyzing data across different time units.

Extracting fields: You saw how to use date_part('field', timestamp) and EXTRACT(field FROM timestamp) to extract fields from dates or timestamps. Both functions serve similar purposes but have different syntaxes. For example, to get the month from a date, you could use:
`SELECT date_part('month', timestamp);`

-
- **Aggregating data by time units**: The lesson highlighted the importance of aggregating data by time units (e.g., by month or year) to analyze trends over time. This involves using functions like date_trunc to truncate dates or timestamps to a specified precision, making it easier to group and compare data.
- **Practical applications**: Through exercises, you applied these concepts to real-world scenarios, such as analyzing the average time to complete service requests by day of the week or finding monthly trends in data by truncating dates to the first of each month.

GENERATE_SERIES

Understanding `generate_series()`: You discovered how to use the `generate_series()` function with date/time data to create a series of timestamps between two points in time, defined by a start and end timestamp, and an interval. This function is particularly useful for filling in gaps in time-series data.
For example:

```
SELECT generate_series('2023-01-01', '2023-01-31', '1 day');
```

-
- **Finding Missing Dates**: You learned to identify missing dates in a dataset by generating a series of all possible dates within a range and then comparing this series to the actual dates present in your data. This technique helps in uncovering any dates with no data recorded.
- **Custom Aggregation Periods**: The lesson included methods for aggregating data over custom time periods, such as every six months or in three-hour intervals, using `generate_series()` to create bins of time. This approach allows for more flexible and detailed analysis of time-based patterns in data.
- **Calculating Averages with Missing Dates**: You explored how to calculate the average number of events per day for each month, including days with no events, by generating a series of all days within a period and left joining this series with your data. This ensures that your averages accurately reflect the entire period, including days with zero events.

When you left 1 day ago, you worked on Introduction to business intelligence for a online movie rental database, chapter 1 of the course Data-Driven Decision Making in SQL. Here is what you covered in your last lesson:

You learned about the importance of aggregating data in SQL for business intelligence, focusing on a case study involving an online movie rental company, MovieNow. Aggregation functions like AVG, SUM, COUNT, MIN, and MAX help summarize vast amounts of data into meaningful insights. For instance:

- The AVG function calculates the average value of a numeric column, such as the average renting price of movies.
- COUNT() is used to count the number of rows in a table or non-NULL values in a column. For example, `SELECT COUNT(*) FROM actors` counts all rows in the 'actors' table.
- DISTINCT is used to remove duplicate values from a result set, allowing for the count of unique entries, such as distinct countries from the customer table.

You also explored how to use SQL to summarize customer information, analyze movie ratings, and examine annual rentals. A key part of summarizing data involves using aliases for better readability of the results, as shown in the query for summarizing ratings for movie 25:

```
SELECT MIN(rating) AS min_rating,
       MAX(rating) AS max_rating,
       AVG(rating) AS avg_rating,
       COUNT(rating) AS number_ratings
FROM renting
WHERE movie_id = 25;
```

Grouping movies

You learned about advanced SQL query techniques to analyze data in more depth. Specifically, you explored how to use the GROUP BY clause to aggregate data into groups based on one or more columns. This is crucial for summarizing data and extracting meaningful insights from large datasets. Here are the key points you covered:

- **Grouping Data**: You saw how to group movies by genre to calculate the average rental price per genre. This involved selecting the genre and using the GROUP BY clause to aggregate data by genre.
- **Aggregation Functions**: You learned to apply aggregation functions like AVG for average, SUM for total, MIN for minimum, and COUNT for counting items within each group. For example, calculating the average price of movies within each genre.
- **Filtering Groups with** HAVING: The HAVING clause was introduced as a way to filter groups based on aggregate functions. This is useful when you want to include only those groups that meet certain conditions, such as having more than a specific number of items.
- **Analyzing Customer Data**: You applied these concepts to analyze customer data, such as finding the date when the first customer account was created in each country and summarizing movie ratings and rentals per customer.

Here's an example query you worked with to analyze customer data:

```
SELECT customer_id,  -- Report the customer_id
       AVG(rating) AS average_rating, -- Report the average rating per customer
       COUNT(rating) AS number_of_ratings, -- Report the number of ratings per customer
       COUNT(*) AS number_of_rentals -- Report the number of movie rentals per customer
```

```
FROM renting
GROUP BY customer_id
HAVING COUNT(*) > 7 -- Select only customers with more than 7 movie rentals
ORDER BY AVG(rating); -- Order by the average rating in ascending order
```

Joining movie ratings with customer data

You learned about the intricacies of SQL JOIN operations, focusing on how to combine data from multiple tables to enrich your dataset. Specifically, you explored:

- The concept of JOIN queries, which allow for the merging of data from more than one table. This is crucial for creating comprehensive datasets that offer more insight than single-table queries.
- The use of LEFT JOIN to combine all rows from a "left" table with matching rows from a "right" table, and how it differs from other types of JOINs by including all rows from the left table regardless of whether there's a match in the right table.
- How to use aliases for tables within your queries to improve readability and manage tables with overlapping column names. For example, assigning the alias c to a customers table allows you to succinctly reference its columns.
- The process of joining tables based on a common identifier, such as customer_id, to merge relevant data. This technique was applied to add customer information to movie rental records, enhancing the dataset with valuable context.

You also practiced writing a SQL query to list movie titles and actor names, ensuring each actor-movie pair is unique. Here's the query you used:

```
SELECT a.name, -- Create a list of movie titles and actor names
       m.title
FROM actsin AS ai
LEFT JOIN movies AS m
ON m.movie_id = ai.movie_id
LEFT JOIN actors AS a
ON a.actor_id = ai.actor_id;
```

Money spent per customer with sub-queries

You learned about using sub-queries in SQL to perform more complex data analysis tasks. Sub-queries, or subsequent SELECT statements, allow you to use the result of one query as a

table in another query. This technique is particularly useful for breaking down complex problems into simpler, more manageable parts.

Key points covered include:

- The basic structure of a sub-query, where you place the first query inside parentheses and give it an alias, making it act like a temporary table for the outer query to use.
- How to calculate the total money spent by each customer on MovieNow rentals by joining tables and using a sub-query to sum up rental fees.
- The importance of grouping results, such as by customer_id or actor nationality, to aggregate data in meaningful ways.

For example, to find the age range of actors from the USA, you used a sub-query to select all actors with US nationality, then grouped the results by gender to find the oldest and youngest actors:

```
SELECT a.gender,
       MIN(a.year_of_birth),
       MAX(a.year_of_birth)
FROM
   (SELECT *
   FROM actors
   WHERE nationality = 'USA') AS a
GROUP BY a.gender;
```

- **Combining SQL Statements**: You combined LEFT JOIN, WHERE, GROUP BY, HAVING, and ORDER BY to create a comprehensive query. This allowed you to augment tables with necessary information from multiple sources, such as linking the 'renting' table with 'actors' through an intermediary 'actsin' table.
- **Filtering and Aggregation**: You filtered customers by gender using WHERE and then grouped results by actor names with GROUP BY. This setup helped in counting how often movies featuring certain actors were watched by a targeted demographic.
- **Calculating Averages and Excluding Nulls**: By including an average rating in your query, you learned to refine your analysis to consider both the popularity and quality of movies. The HAVING clause was used to exclude actors with movies that had no ratings, demonstrating the difference between WHERE (pre-aggregation filtering) and HAVING (post-aggregation filtering).
- **Ordering Results**: The final query was ordered to show actors with the highest average ratings first, and within those, the ones with the most views, revealing insights into customer preferences.

- **Application to Different Queries**: You applied similar techniques to identify favorite movies for customers born in the 70s and explore actor popularity in Spain, adjusting your approach based on the question at hand.
- **KPIs per Country**: You prepared a report with key performance indicators (KPIs) for each country, focusing on the total number of movie rentals, average movie ratings, and total revenue since 2019.

For example, to identify favorite actors for male customers, your query might have looked like this:

```
SELECT a.name, COUNT(*) AS views, AVG(r.rating) AS average_rating
FROM renting r
LEFT JOIN actsin ai ON r.movie_id = ai.movie_id
LEFT JOIN actors a ON ai.actor_id = a.actor_id
WHERE r.customer_gender = 'male'
GROUP BY a.name
HAVING average_rating IS NOT NULL
ORDER BY average_rating DESC, views DESC;
```

Nested Queries

- Definition and Usage: A nested query is a SELECT statement within the WHERE or HAVING clause of another SELECT statement. It's executed first, and its result is used by the outer query.
- Identifying Disappointed Customers: You saw how to identify customers who rated a movie 3 or lower by nesting a query that selects customer IDs from a ratings table within another query that selects customer names based on those IDs.
- Countries with Earlier Accounts than Austria: By placing a subquery in the HAVING clause, you learned to list countries where the first account was created earlier than the first account in Austria.
- Actors in a Specific Movie: You explored multi-level nesting by first selecting a movie ID, then using that ID to find actor IDs, and finally selecting actor names based on those IDs.

Additionally, you practiced writing nested queries to solve practical problems:

- Listing movies with more than 5 views by excluding seldom-watched movies for advertising purposes.
- Reporting frequent customers who rented more than 10 movies, utilizing the `IN` statement and `HAVING` clause for filtering based on a count condition.

Here's an example query you worked with:

```
SELECT *
```

```
FROM customers
WHERE customer_id IN
        (SELECT customer_id
        FROM renting
        GROUP BY customer_id
        HAVING COUNT(*) > 10);
```

Correlated nested queries

- Definition and Use: A correlated nested query is used when you need to perform a query that references columns from an outer query within its WHERE clause. This is useful for detailed, row-specific operations.
- Examples: You saw how to identify movies rented more than 5 times using a correlated query by linking the movies and renting tables. This involved comparing the count of rentals per movie to a specified number, demonstrating the query's ability to loop through each movie and apply the condition dynamically.
- Practical Applications:
    - Analyzing customer behavior to target those who rented fewer than 5 movies for a new advertising campaign.
    - Identifying customers who gave low ratings, specifically those with a minimum rating below 4.
    - Reporting on movies that received significant attention, either through a high number of ratings or an average rating above 8.

Here's a snippet of a correlated query you worked with:

```
SELECT m.movie_id, m.title
FROM movies m
WHERE 5 < (
   SELECT COUNT(*)
   FROM renting r
   WHERE r.movie_id = m.movie_id);
```

Queries with EXISTS

- The purpose of the EXISTS function, which checks if the result of a nested query is non-empty, returning TRUE if at least one row exists, and FALSE otherwise. This helps in selecting rows from an outer query based on the existence of related data.

- How to use EXISTS in a practical example, where you identified movies with at least one rating. You learned that the EXISTS function allows for a simplified subquery syntax, typically using SELECT *, since the presence of rows, not their content, is what matters.

```
SELECT movie_id
FROM movies
WHERE EXISTS
    (SELECT *
    FROM ratings
    WHERE movies.movie_id = ratings.movie_id);
```

- The NOT EXISTS variant, which selects rows when a subquery returns empty. This was illustrated through selecting movies without any ratings, demonstrating how to invert the logic of EXISTS for different use cases.

Additionally, you applied these concepts in exercises aimed at identifying active customers and analyzing the diversity of actors in comedies, further solidifying your understanding of how EXISTS can be leveraged to extract meaningful insights from relational databases.

The goal of the next lesson is to learn how to use advanced SQL queries, including UNION and INTERSECT operators, to combine and analyze data from multiple tables.

OLAP: CUBE operator

- UNION Operator: You discovered that UNION combines the results of two or more SELECT statements by removing duplicates. Each SELECT statement within the UNION must have the same number of columns, and those columns must have similar data types. For example, if you have two tables where one lists movies with a renting price higher than 2.8 and the other lists Action and Adventure movies, UNION would combine these lists into one, excluding any duplicates.
- INTERSECT Operator: You learned that INTERSECT returns only the rows that exist in both SELECT statement results. It's a great way to find common elements between two sets of data. For instance, if you use INTERSECT on the same tables as in the UNION example, you would get movies that are both priced above 2.8 and belong to the Action and Adventure genre.

Additionally, you applied these concepts in exercises, such as identifying young actors not from the USA and finding dramas with high ratings, further enhancing your ability to make data-driven decisions using SQL.

The goal of the next lesson is to understand the role of OLAP in enhancing business intelligence through interactive data analysis, summarization, and visualization, focusing on the use of the CUBE operator in SQL for creating detailed pivot tables.

OLAP: CUBE operator

- The concept of OLAP and its role in data analysis.
- How the CUBE operator works in conjunction with the GROUP BY clause to perform aggregations on multiple levels. For instance, it allows for aggregation by each combination of specified columns, as well as by each column independently, and for the total aggregation across all rows.
- The practical application of the CUBE operator to create a pivot table from a dataset, demonstrated with the `rentings_extended` table. This table included data aggregated by country and genre, showing the number of movie rentals.

You also practiced writing SQL queries using the CUBE operator to:

- Extract aggregated data for creating pivot tables, such as the total number of customers by gender and country.
- Analyze movie data by listing the number of movies for different genres and release years.
- Prepare a report comparing the average rating of movies across countries and genres.

Here's an example SQL query you used to aggregate data using the CUBE operator:

```
SELECT gender, country, COUNT(*)
FROM customers
GROUP BY CUBE (gender, country)
ORDER BY country;
```

This query demonstrates how to use the CUBE operator to aggregate customer data by gender and country, showcasing its utility in extracting complex aggregated data for analysis and reporting purposes.

ROLLUP

- **Understanding ROLLUP**: It is used with the GROUP BY statement to create subtotals and grand totals. The order of columns within the ROLLUP function is crucial as it determines the hierarchy of aggregation.
- **Syntax and Usage**: You saw how to use ROLLUP to aggregate data across multiple levels, such as aggregating movie rentals by country and genre, then by country alone, and finally, the total number of rentals.
- **Practical Application**: Through exercises, you applied ROLLUP to count the total number of customers, as well as the number of customers by country and by gender. Additionally, you explored how to analyze genre preferences across countries by averaging ratings and counting movie rentals.

```
SELECT country,
    gender,
    COUNT(*)
FROM customers
GROUP BY ROLLUP (country, gender)
ORDER BY country, gender;
```

This query demonstrates how to use ROLLUP to count customers by country and gender, showcasing the operator's ability to provide insights at different aggregation levels.

The goal of the next lesson is to learn how to use the GROUPING SETS operator in SQL for advanced data aggregation, enabling more complex and detailed analysis in a single query.

GROUPING SETS

- **Understanding GROUPING SETS**: You discovered that GROUPING SETS is an OLAP (Online Analytical Processing) extension that enables flexible data aggregation. It's like performing multiple GROUP BY operations in one query, allowing for a more detailed analysis of data.

- **Syntax and Usage**: Through examples, you learned how to use the GROUPING SETS syntax. For instance, GROUP BY GROUPING SETS ((column1, column2), (column3), ()) allows for aggregation by different column combinations and the total aggregation.
- **Practical Application**: You applied this knowledge to analyze movie rental data, exploring different levels of aggregation such as the number of movie rentals by country and genre, by country, by genre, and the total number of rentals. This was demonstrated with the following code snippet:

```
SELECT country, genre, COUNT(*)

FROM renting_extended

GROUP BY GROUPING SETS ((country, genre), (country), (genre), ());
```

- This query showcases how to extract detailed insights from the data, such as the total number of rentals, as well as more granular insights by country and genre.
- **Exploring Further**: Exercises included queries on customer ratings and movie actor demographics, reinforcing the concept of GROUPING SETS for diverse analytical needs. For example, you explored the diversity of actor nationalities and gender distribution in films, and analyzed customer ratings by country and gender, enhancing your ability to derive meaningful insights from complex datasets.

This lesson equipped you with the knowledge to use GROUPING SETS for sophisticated data analysis, enabling you to perform detailed and flexible aggregations in SQL.

The goal of the next lesson is to learn how to analyze movie genres and actors using SQL to make informed investment decisions for new movies.

Plots

- **Scatter Plots Basics**: Understanding that scatter plots are used to visualize the relationship between two continuous variables. You saw how plotting home prices against area in Los Angeles County could reveal patterns in the data.
- **Logarithmic Scales**: The application of logarithmic scales to both axes of a scatter plot to spread out clustered data points, making the plot easier to interpret.
- **Correlation**: The concept of correlation was introduced, explaining how it measures the strength and direction of a linear relationship between two variables. You learned about positive, negative, and no correlation through theoretical datasets and the importance of a straight line in identifying linear relationships.

- **Trend Lines**: How adding a straight or curved trend line to a scatter plot can help in understanding the nature of the relationship between variables. For example, a linear increase in the logarithm of the area corresponds to a linear increase in the logarithm of the price.

Here's an example code snippet to plot a basic scatter plot in Python using matplotlib:

```
import matplotlib.pyplot as plt


# Sample data
x = [1, 2, 3, 4, 5]  # Example x-axis values
y = [2, 3, 5, 7, 11]  # Example y-axis values


plt.scatter(x, y)
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Scatter Plot Example')
plt.show()
```

Avoiding common errors data communication and presentation

You learned about delivering compelling oral presentations, focusing on the importance of preparation, practice, and engaging with your audience effectively. Key points included:

- Practicing Your Presentation: Emphasized the value of rehearsing with your actual slides and speaking out loud to become familiar with your content. This helps identify and eliminate filler words and awkward phrasing, and improve transitions between slides.
- Handling Emotions and Audience Engagement: Highlighted the need to manage your emotions to maintain audience confidence in your content. Techniques such as eye contact, interactivity, and avoiding phrases like "as you know" can keep the audience engaged and build a relationship with them.
- Presentation Timing and Pace: Stressed the importance of keeping within the allocated time to avoid losing the audience's attention. Making pauses and varying speaking pace can help emphasize key points and give the audience time to absorb the information.

- Encouraging Questions: Encouraged opening up for questions during or at the end of the presentation to demonstrate openness to feedback and interest in the audience's understanding.

By applying these techniques, you're equipped to deliver presentations that are not only informative but also engaging and persuasive.

- Data as the New Oil: Just as oil undergoes various processes from extraction to reaching the end consumer, data is processed and moved through pipelines from collection to utilization.
- Spotflix Example: You explored how Spotflix uses data pipelines to handle data from different sources, such as user actions and internal systems, and processes it for various needs like recommendations.

ETL Process: The term ETL, standing for Extract, Transform, Load, was introduced as a popular framework for designing data pipelines. It highlights the sequential steps of extracting data from sources, transforming it to fit operational needs, and loading it into a destination for storage and analysis.

```
# Example of an ETL process

extract_data()

transform_data()

load_data()
```

- Exercise on Building a Pipeline: You practiced ordering steps to build a pipeline for generating a "Weekly Playlist" for a user, emphasizing the importance of understanding the user's preferences and finding similar tastes.

Cloud computing

You learned about the significance of cloud computing in modern data processing and management. Cloud computing allows companies to rent servers on-demand, offering a cost-effective alternative to maintaining physical servers on-premises. This approach not only

reduces the need for physical space but also optimizes resource usage by scaling processing power according to demand. Key points covered include:

- **Cost Optimization**: Cloud computing helps avoid the waste of resources by allowing companies to use and pay for only the processing power they need, when they need it.
- **Global Accessibility and Reduced Latency**: By utilizing servers located closer to the end-users, companies can ensure lower latency and better performance for their applications.
- **Database Reliability**: Cloud services enhance data reliability through geographical data replication, safeguarding against data loss during disasters.
- **Major Cloud Providers**: You learned about the three major cloud providers: Amazon Web Services (AWS), Microsoft Azure, and Google Cloud, along with their key services:
    - AWS: S3 for storage, EC2 for computation, and RDS for databases.
    - Azure: Blob Storage, Virtual Machines, and SQL Database.
    - Google Cloud: Cloud Storage, Compute Engine, and Cloud SQL.
- **Multicloud Strategy**: The lesson highlighted the advantages of using multiple cloud providers, such as cost optimization, reduced vendor reliance, and increased disaster resilience. However, it also mentioned potential challenges like vendor lock-in and increased complexity in security and governance.

You also explored the concept of multicloud computing, understanding its benefits and potential drawbacks through exercises that challenged you to classify cloud services and evaluate the implications of a multicloud approach for a hypothetical company, Spotflix.

```
# Example of cloud service classification

services = {"AWS S3": "Storage", "Azure Virtual Machines": "Computing", "Google Cloud SQL": "Database"}
```

This lesson equipped you with a foundational understanding of cloud computing's role in data engineering, preparing you for more advanced topics in data processing and management.

You learned about administering a Databricks workspace and the roles involved in managing it. Here's a recap of the key points:

- **Account Admin Responsibilities**:
    - The Account Admin acts as the highest-level administrator, overseeing all Databricks activities.
    - They manage the Account Console, where they can create workspaces, add users, govern access, and monitor billing.
    - They ensure the Databricks environment is set up correctly and efficiently.
- **Workspace Admin Responsibilities**:
    - Workspace Admins focus on specific workspaces, ensuring teams have the necessary access and capabilities.
    - They handle workspace-specific settings and monitor activities within those workspaces.
- **Connecting to External Systems**:
    - **Partner Connect**: Facilitates integration with external technologies like BI tools and data ingestion platforms, enhancing Databricks' connectivity.
    - **Marketplace**: Allows discovery and acquisition of third-party datasets, enriching your data catalogs and analytics capabilities.

These components are crucial for effective data management and seamless integration within the Databricks Lakehouse platform.

- **Databricks and Apache Spark**: Databricks is built on Apache Spark, an open-source framework that efficiently distributes work across computing resources, supporting multiple languages and use cases.
- **Cluster Management**: Clusters in Databricks process data and perform analytics. They are collections of resources managed by Databricks, which can be created using either classic or serverless architecture.
    - **Classic Architecture**: Offers control over resources but has slower startup times.
    - **Serverless Architecture**: Provides faster startup times and improved performance, though it offers less direct control over resources.
- **Cluster Designs**:

    - **Single-node Clusters**: Suitable for small datasets and single-machine frameworks like pandas.
    - **Multi-node Clusters**: Ideal for large datasets, utilizing Spark to distribute workloads.
- **Databricks Runtime**: Each cluster runs on the Databricks Runtime, which includes an optimized Spark version, Photon engine for fast SQL queries, and necessary libraries and APIs.

Merging tables in databriks code:

```
MERGE INTO employee AS e USING employee_new AS n

  ON e.AGENT_ID = n.AGENT_ID

  WHEN MATCHED THEN UPDATE SET e.POSTAL_CODE = n.POSTAL_CODE
```